SE 403 – Hướng dẫn thực hiện Group Project

Oct 31, 2026 · @ManNguyen

Tài liệu này đi kèm bản mô tả 5 đề tài. Sinh viên được dùng AI ở mọi bước, nhưng mọi thành viên phải hiểu và tự giải thích được từng quyết định trong bài nộp; điểm số phụ thuộc vào khả năng giải thích đó.

1. Quy định sử dụng AI

AI là trợ lý, không phải tác giả: nhóm chịu trách nhiệm với mọi dòng trong bài nộp, kể cả những dòng AI viết. Tiêu chí duy nhất: **bất kỳ thành viên nào cũng phải giải thích được bất kỳ nội dung nào** khi giảng viên hỏi.

Được khuyến khích	Không được phép
Hỏi AI giải thích khái niệm (tactic, pattern, ATAM) và cho ví dụ	Dán nguyên output AI vào báo cáo mà không đọc, kiểm tra, chỉnh sửa
Brainstorm stakeholders, QA scenarios, rủi ro rồi nhóm tự chọn lọc	Để AI tự quyết định kiến trúc mà nhóm không so sánh phương án
Nhờ AI đóng vai phản biện (“tìm điểm yếu của thiết kế này”)	Bịa số liệu đo, kết quả load test hoặc trích dẫn do AI tạo ra
Sinh code khung, script test, Dockerfile, sơ đồ PlantUML/Mermaid	Nộp code mà không thành viên nào giải thích được
Rà soát ngữ pháp, trình bày tài liệu	Che giấu việc dùng AI

AI Log (bắt buộc, nộp kèm mỗi mốc). Mỗi lần dùng AI cho nội dung đưa vào bài nộp, ghi một dòng:

Ngày	Thành viên	Công cụ	Mục đích / prompt tóm tắt	AI đề xuất gì	Nhóm đã giữ / sửa / bỏ gì và vì sao
05/11	Nguyễn V. A	ChatGPT	Gợi ý QA scenario cho availability	8 scenario	Giữ 3, sửa response measure từ “nhanh” thành “≤ 30 giây”; bỏ 5 vì không đúng bối cảnh

Cột cuối là quan trọng nhất: nó cho thấy nhóm đã tư duy, không chỉ sao chép.

Quy tắc “3 câu hỏi” trước khi giữ output AI. Nhóm chỉ đưa nội dung vào bài khi trả lời được:

1. Nội dung này giải quyết QA scenario / yêu cầu nào của đề tài chúng ta?

2. Phương án thay thế là gì, và vì sao chúng ta không chọn nó?
3. Nếu bỏ nội dung này đi thì điều gì xấu sẽ xảy ra?

Kiểm tra hiểu bài. Giảng viên sẽ vấn đáp ngẫu nhiên từng thành viên ở mỗi mốc (5–10 phút/nhóm), chỉ vào một ADR, một sơ đồ hoặc một đoạn code bất kỳ. Thành viên không giải thích được phần do chính mình phụ trách sẽ bị giảm hệ số đóng góp; cả nhóm không giải thích được thì nội dung đó được 0 điểm.

2. Quy trình chung cho cả 5 đề tài

Quy trình gồm 15 bước chia theo 5 mốc nộp. Mỗi bước có ba phần: **Làm** (việc nhóm phải tự làm), **AI hỗ trợ** (cách dùng AI hợp lệ, kèm prompt mẫu) và **Tự kiểm tra** (câu hỏi mà mỗi thành viên phải trả lời được không cần tài liệu).

Giai đoạn 0 – Khởi động (tuần 1–2)

1. **Lập nhóm và phân vai.**
   - Làm: chọn đề tài; phân vai Lead Architect, Requirements Owner, Prototype Lead, QA/Test Lead, Documentation Owner (mỗi người vẫn tham gia mọi phần); tạo Git repo với các thư mục `docs/`, `adr/`, `diagrams/`, `src/`, `ai-log.md`.
   - AI hỗ trợ: "Gợi ý cấu trúc repository cho một dự án thiết kế kiến trúc có ADR và sơ đồ C4 dạng code."
   - Tự kiểm tra: Vai trò của tôi chịu trách nhiệm sản phẩm nào ở mỗi mốc?
2. **Đọc hiểu đề tài.**
   - Làm: mỗi thành viên tự đọc và viết 5 câu tóm tắt bài toán bằng lời của mình, sau đó cả nhóm so sánh và thống nhất.
   - AI hỗ trợ: "Tôi sẽ mô tả bài toán [dán đề]. Hãy đặt 10 câu hỏi làm rõ yêu cầu mà một kiến trúc sư sẽ hỏi khách hàng." – nhóm tự trả lời, ghi giả định (assumption) vào tài liệu.
   - Tự kiểm tra: Điều gì làm bài toán này khó về mặt kiến trúc, khác với một ứng dụng CRUD thông thường?

Giai đoạn 1 – Architectural Drivers (→ M1, tuần 4)

3. **Xác định business goals và stakeholders.**
   - Làm: liệt kê 3–5 business goal đo được; bảng stakeholder – mối quan tâm – QA họ quan tâm nhất.
   - AI hỗ trợ: "Với stakeholder là [X] trong hệ thống [Y], họ thường lo ngại điều gì về hệ thống?"
   - Tự kiểm tra: Hai stakeholder nào có mối quan tâm xung đột nhau?

4. **Use cases và constraints.**
   - Làm: 6–10 use case chính (dạng ngắn), ràng buộc kỹ thuật/kinh doanh/pháp lý (ngân sách, thời gian, luật).
   - AI hỗ trợ: kiểm tra use case thiếu: "Danh sách use case này còn thiếu luồng ngoại lệ quan trọng nào?"
   - Tự kiểm tra: Constraint khác requirement ở điểm nào? Cho một ví dụ trong đề tài.
5. **Viết Quality Attribute Scenarios.**
   - Làm: ≥ 6 scenario đủ 6 phần (source, stimulus, artifact, environment, response, response measure), phủ ít nhất 4 QA khác nhau.
   - AI hỗ trợ: nhóm viết trước, sau đó hỏi "Scenario này có đo lường và kiểm thử được không? Chỉ ra phần mơ hồ."
   - Tự kiểm tra: Con số trong response measure lấy từ đâu? Làm sao để kiểm chứng nó trong prototype?
6. **Lập Utility Tree và chọn ASR.**
   - Làm: xếp hạng mỗi scenario theo (Business importance, Technical risk) H/M/L; chọn 3–5 ASR có hạng (H,H).
   - AI hỗ trợ: giải thích cách xếp hạng; không để AI xếp hạng thay nhóm.
   - Tự kiểm tra: Vì sao scenario X được (H,H) còn scenario Y chỉ (H,L)?

Giai đoạn 2 – Thiết kế kiến trúc (→ M2, tuần 8)

7. **Chọn tactics cho từng ASR.**
   - Làm: với mỗi ASR, liệt kê 2–3 tactic ứng viên (theo catalog của Bass et al.), chọn một và ghi lý do.
   - AI hỗ trợ: "Liệt kê các tactic cho availability và ưu/nhược điểm của từng tactic trong bối cảnh [ASR cụ thể]."
   - Tự kiểm tra: Tactic đã chọn ảnh hưởng tiêu cực tới QA nào khác?
8. **Chọn architectural style/pattern tổng thể.**
   - Làm: so sánh ít nhất 2 phương án (ví dụ modular monolith vs microservices) bằng bảng đánh giá theo từng ASR.
   - AI hỗ trợ: yêu cầu AI đóng vai "devil's advocate" phản bác lựa chọn của nhóm.
   - Tự kiểm tra: Với quy mô nhóm 5 người và 15 tuần, lựa chọn này có thực tế không?
9. **Vẽ các architecture view.**
   - Làm: C4 Level 1 (Context), Level 2 (Container), Level 3 (Component) cho ít nhất 1 container quan trọng; Deployment view; 1–2 sequence diagram cho luồng ASR. Mỗi sơ đồ có legend và đoạn mô tả.

- AI hỗ trợ: sinh code PlantUML/Mermaid/Structurizr từ mô tả của nhóm; nhóm kiểm tra từng mũi tên.
- Tự kiểm tra: Chỉ vào một mũi tên bất kỳ: giao thức gì, đồng bộ hay bất đồng bộ, nếu đích bị lỗi thì sao?

10. **Viết ADR.**
    - Làm: ≥ 5 ADR theo mẫu: Title, Status, Context, Options considered, Decision, Consequences (tích cực và tiêu cực).
    - AI hỗ trợ: gợi ý các phương án còn bỏ sót; rà soát ngôn ngữ. Phần Decision và Consequences do nhóm tự viết.
    - Tự kiểm tra: Nếu bối cảnh thay đổi (ví dụ tải tăng 10 lần), ADR nào sẽ bị thay thế?

Giai đoạn 3 – Đánh giá kiến trúc (→ M3, tuần 11)

11. **Thực hiện ATAM rút gọn.**
    - Làm: với 3 ASR ưu tiên cao nhất, phân tích cách kiến trúc đáp ứng; xác định sensitivity points, trade-off points, risks, non-risks. Nhóm trình bày cho một nhóm khác (peer review) và ghi lại phản hồi.
    - AI hỗ trợ: "Với kiến trúc sau [mô tả], hãy đặt các câu hỏi ATAM mà một evaluation team sẽ hỏi cho scenario [X]."
    - Tự kiểm tra: Phân biệt sensitivity point và trade-off point bằng ví dụ trong chính thiết kế của nhóm.
12. **Cải tiến lên Architecture v2.**
    - Làm: sửa thiết kế để xử lý ít nhất 2 risk; cập nhật ADR (đổi status "Superseded" nếu thay quyết định).
    - Tự kiểm tra: Thay đổi từ v1 sang v2 là gì và risk nào đã được giảm?

Giai đoạn 4 – Prototype (→ M4, tuần 14)

13. **Xây dựng architectural skeleton.**
    - Làm: hiện thực các thành phần và kết nối chính theo Container view; cấu trúc code phải ánh xạ được về sơ đồ; chạy được bằng một lệnh `docker compose up`.
    - AI hỗ trợ: sinh code khung, Dockerfile, mock service; mỗi file AI sinh ra phải có một thành viên review và ghi tên trong AI log.
    - Tự kiểm tra: Chỉ ra trong code chỗ hiện thực tactic X. Nếu xóa đoạn đó, scenario nào thất bại?
14. **Kiểm chứng quality attributes.**
    - Làm: với ≥ 2 ASR, thiết kế thí nghiệm (load test, fault injection, security test), chạy, thu số liệu thật, so với response measure.

- AI hỗ trợ: viết script k6/JMeter/Locust, script tắt container; giải thích số liệu. Tuyệt đối không dùng số liệu do AI "ước tính".
- Tự kiểm tra: Nếu kết quả không đạt, nguyên nhân do kiến trúc hay do môi trường test?

Giai đoạn 5 – Hoàn thiện (→ M5, tuần 15)

15. **Hoàn thiện SAD, slide và chuẩn bị vấn đáp.**
    - Làm: Software Architecture Document gồm: giới thiệu, drivers, views, ADR, kết quả đánh giá, kết quả kiểm chứng, bài học, AI log. Mỗi thành viên trình bày một phần và được hỏi phần của người khác.
    - AI hỗ trợ: "Đóng vai giảng viên SAD, hỏi tôi 10 câu khó về thiết kế này và chấm câu trả lời của tôi." – cách luyện vấn đáp hiệu quả nhất.
    - Tự kiểm tra: Nếu làm lại từ đầu, nhóm sẽ thay đổi quyết định nào và vì sao?

3. Hướng dẫn riêng – Đề tài 1: MediLink

Trọng tâm của đề tài này là **bảo mật dữ liệu y tế, tích hợp HIS không đồng nhất và chống đặt trùng lịch**. Các bước dưới đây bổ sung cho quy trình chung (số trong ngoặc là bước tương ứng).

1. **Phân tích pháp lý và dữ liệu nhạy cảm (bước 4).** Lập bảng phân loại dữ liệu: công khai, cá nhân, cá nhân nhạy cảm (sức khỏe). Đọc tóm tắt Nghị định 13/2023/NĐ-CP và rút ra ⩾ 3 constraint kỹ thuật.
2. **Scenario bắt buộc phải có (bước 5):** (a) 50 bệnh nhân cùng đặt một slot; (b) HIS của cơ sở Huế ngừng hoạt động 2 giờ; (c) lễ tân cố xem hồ sơ bệnh nhân không thuộc cơ sở mình; (d) tích hợp HIS thứ 4.
3. **Thiết kế lớp tích hợp (bước 7–8).** So sánh: gọi trực tiếp từng HIS, Adapter per HIS, Anti-Corruption Layer + canonical data model (tham khảo FHIR resource `Appointment`, `Patient`). Chọn đồng bộ hay bất đồng bộ cho từng luồng.
4. **Thiết kế chống double-booking (bước 7).** So sánh pessimistic lock, optimistic lock (version), unique constraint trên (doctor_id, slot), distributed lock (Redis). Thêm idempotency key cho request đặt lịch.
5. **Thiết kế bảo mật (bước 9).** Vẽ trust boundary trên Context/Container view; RBAC + kiểm tra thuộc cơ sở (ABAC đơn giản); mã hóa trường nhạy cảm; audit log chỉ ghi thêm (append-only).
6. **ADR gợi ý:** chiến lược tích hợp HIS; cơ chế chống double-booking; mô hình phân quyền; nơi lưu file kết quả xét nghiệm; cách gửi nhắc lịch (queue + worker).

7. **Prototype (bước 13–14).** Dựng 2 mock HIS (một REST/JSON, một XML) bằng WireMock hoặc Express; viết script bắn 50 request đồng thời vào cùng slot rồi đếm số booking thành công (kỳ vọng = 1); tắt một mock HIS và chứng minh cơ sở khác vẫn đặt được (circuit breaker); test truy cập trái quyền trả về 403 và có dòng audit.

Prompt AI gợi ý: "So sánh optimistic và pessimistic locking cho bài toán đặt lịch khám với 50 request đồng thời; khi nào mỗi cách thất bại?" · "Giải thích Anti-Corruption Layer bằng ví dụ hai hệ thống dùng mã bệnh nhân khác nhau."

Sinh viên phải tự giải thích được:

- Vì sao chỉ unique constraint trong database có thể chưa đủ khi có nhiều instance và cache?
- Khi HIS trả lỗi timeout, hệ thống làm gì với lịch hẹn đang chờ xác nhận?
- Làm sao chứng minh audit log không bị sửa?

Lỗi thường gặp: chỉ nói "dùng HTTPS và JWT" cho toàn bộ phần security; vẽ mỗi HIS thành một microservice mà không giải thích vì sao.

4. Hướng dẫn riêng – Đề tài 2: ShopHub

Trọng tâm của đề tài này là **phân rã hệ thống, giao dịch phân tán và chịu tải đột biến khi flash sale**.

1. **Ước lượng tải (bước 4–5).** Từ 200.000 khách hàng, tự tính số người dùng đồng thời bình thường và khi flash sale; ghi rõ các giả định (tỉ lệ active, số request/người). Con số này là cơ sở cho response measure.
2. **Scenario bắt buộc:** (a) 1.000 người mua 100 sản phẩm trong 10 giây; (b) cổng thanh toán trả lỗi sau khi đã trừ tồn kho; (c) dịch vụ gợi ý sập; (d) thêm đơn vị vận chuyển mới.
3. **Phân rã theo bounded context (bước 8).** Tổ chức một buổi Event Storming 2 giờ (giấy note hoặc Miro): liệt kê domain event → command → aggregate → nhóm thành context (Catalog, Ordering, Inventory, Payment, Shipping, Seller, Review). Chụp ảnh kết quả đưa vào báo cáo.
4. **Quyết định monolith hay microservices (bước 8, ADR bắt buộc).** Đánh giá theo ASR, quy mô đội, chi phí vận hành. Phương án lai (modular monolith + tách riêng Inventory/Flash-sale) hoàn toàn chấp nhận được nếu lập luận tốt.
5. **Thiết kế luồng đặt hàng (bước 7, 9).** Vẽ sequence diagram Saga (chọn orchestration hay choreography và giải thích); mỗi bước có compensation; dùng Outbox để đảm bảo không mất event.
6. **Thiết kế flash sale (bước 7).** So sánh: trừ tồn kho trong DB có lock; atomic decrement trên Redis; hàng đợi đặt hàng (queue-based load leveling). Thêm rate limiting, CDN/cache cho trang sản phẩm.

7. **ADR gợi ý:** kiểu kiến trúc tổng thể; Saga orchestration vs choreography; công nghệ tìm kiếm (DB full-text vs Elasticsearch); chiến lược cache và invalidation; database-per-service hay shared database.
8. **Prototype (bước 13–14).** Hiện thực Ordering, Inventory, Payment (mock có thể cấu hình tỉ lệ lỗi) và message broker (RabbitMQ/Kafka); chạy k6 kịch bản flash sale, báo cáo throughput, p95 và số đơn vượt tồn kho; bật tỉ lệ lỗi thanh toán 30% và chứng minh tồn kho được hoàn lại đúng.

Prompt AI gợi ý: "Hướng dẫn tôi chạy một buổi Event Storming cho sàn TMĐT, bước nào làm gì?" · "Vì sao dual-write (ghi DB rồi publish event) gây mất dữ liệu, và Outbox giải quyết thế nào?"

Sinh viên phải tự giải thích được:

- Vì sao không dùng một transaction 2PC xuyên suốt các service?
- Nếu Redis sập giữa flash sale, số tồn kho được khôi phục từ đâu?
- Eventual consistency ảnh hưởng thế nào đến trải nghiệm người mua, và nhóm xử lý băng giao diện ra sao?

Lỗi thường gặp: tách 10+ microservice nhưng dùng chung một database; load test trên laptop rồi kết luận "hệ thống chịu được 5.000 req/s" mà không nêu môi trường.

5. Hướng dẫn riêng – Đề tài 3: GoFood

Trọng tâm của đề tài này là **kiến trúc hướng sự kiện, dữ liệu vị trí thời gian thực và độ tin cậy của trạng thái đơn hàng**.

1. **Tính tải vị trí (bước 5).** 3.000 tài xế × 1 bản tin / 3 giây = 1.000 msg/giây; tự tính dung lượng lưu trữ một ngày và quyết định có cần lưu toàn bộ lịch sử vị trí hay không.
2. **Scenario bắt buộc:** (a) 12h trưa, 800 đơn trong 5 phút; (b) tài xế mất sóng 45 giây khi đang giao; (c) hai dispatcher instance cùng gán một đơn cho hai tài xế; (d) mở rộng sang Huế.
3. **Mô hình hóa trạng thái đơn (bước 9).** Vẽ state machine diagram: CREATED → SEARCHING_DRIVER → ASSIGNED → PICKED_UP → DELIVERED / CANCELLED / FAILED; ghi rõ ai được phép chuyển trạng thái và timeout cho từng trạng thái.
4. **Thiết kế event-driven (bước 8).** Xác định topic/event (LocationUpdated, OrderCreated, DriverAssigned…), producer và consumer; so sánh Kafka, RabbitMQ, Redis Streams cho từng loại event; chọn delivery semantics (at-least-once + idempotent consumer).
5. **Thiết kế matching và geo-index (bước 7).** So sánh Redis GEO, PostGIS, geohash/H3 grid; phân vùng theo khu vực để scale theo thành phố.

6. **Kênh real-time tới client (bước 9).** So sánh polling, Server-Sent Events, WebSocket; xử lý kết nối lại và scale WebSocket qua nhiều instance (sticky session hoặc pub/sub backplane).
7. **ADR gợi ý:** message broker; công nghệ geo-index; WebSocket vs SSE; cơ chế chống gán trùng (distributed lock / conditional update); chiến lược phân vùng dữ liệu theo thành phố.
8. **Prototype (bước 13–14).** Viết driver simulator (Python/Node) sinh ≥ 500 tài xế di chuyển trên bản đồ Đà Nẵng; dịch vụ matching; web map (Leaflet) hiển thị real-time; đo độ trễ end-to-end từ lúc gửi vị trí đến lúc hiển thị; `docker stop` một instance dispatcher giữa chừng và chứng minh không mất hoặc gán trùng đơn.

Prompt AI gợi ý: "Viết script Python giả lập N tài xế di chuyển ngẫu nhiên trong một bounding box và publish vị trí lên MQTT/Kafka; giải thích từng phần." · "Khi nào at-least-once delivery gây xử lý trùng, và làm sao thiết kế consumer idempotent?"

Sinh viên phải tự giải thích được:

- Vì sao vị trí tài xế và trạng thái đơn cần yêu cầu nhất quán khác nhau?
- Nếu broker chậm, điều gì xảy ra với bản đồ của khách hàng? Nhóm chấp nhận mất gì?
- Làm sao mở rộng sang thành phố mới chỉ bằng cấu hình?

Lỗi thường gặp: lưu mọi cập nhật vị trí vào database quan hệ rồi query khoảng cách bằng công thức tính toàn bảng; không xử lý trường hợp không tìm được tài xế.

6. Hướng dẫn riêng – Đề tài 4: EduCore

Trọng tâm của đề tài này là **multi-tenancy, khả năng mở rộng bằng plugin và tải đột biến có lịch trước trong giờ thi**.

1. **Phân tích khác biệt giữa các tenant (bước 3–4).** Phỏng vấn ít nhất 2 giảng viên hoặc cán bộ đào tạo (có thể là giảng viên trong trường) về cách tính điểm, quy chế thi; lập bảng "những gì giống nhau / khác nhau giữa các trường". Đây là cơ sở xác định variation point.
2. **Scenario bắt buộc:** (a) 8.000 sinh viên nộp bài trong phút cuối; (b) server restart khi đang thi; (c) lỗi cấu hình khiến query thiếu điều kiện tenant; (d) trường mới muốn công thức điểm 10% chuyên cần + 30% giữa kỳ + 60% cuối kỳ; (e) onboard tenant mới trong 1 ngày.
3. **Chọn mô hình multi-tenancy (bước 8, ADR bắt buộc).** Lập bảng so sánh database-per-tenant, schema-per-tenant, shared schema + tenant_id theo: mức cách ly, chi phí, độ phức tạp migration, khả năng backup/restore từng tenant. Có thể chọn mô hình lai (tenant lớn có DB riêng).

4. **Cơ chế cách ly (bước 7, 9).** Xác định tenant từ đâu (subdomain, token claim); áp tenant filter ở tầng nào (middleware, ORM global filter, Row-Level Security của PostgreSQL) – và vì sao không dựa vào lập trình viên nhớ thêm `WHERE tenant_id`.
5. **Kiến trúc plugin (bước 8).** Định nghĩa microkernel: core gồm gì, extension point nào (grading formula, hành động sau nộp bài, widget dashboard); plugin chạy in-process hay qua webhook/sandbox; ai kiểm duyệt plugin.
6. **Thiết kế cho giờ thi (bước 7).** Autosave phía client (local storage) + gửi định kỳ; ghi bài vào queue trước rồi worker lưu DB; pre-scale theo lịch thi; tách đường đọc (đề thi, cache) và đường ghi.
7. **ADR gợi ý:** mô hình multi-tenancy; cơ chế xác định tenant; kiến trúc plugin; cách định nghĩa công thức điểm (rule engine / DSL / cấu hình JSON); chiến lược lưu bài thi chống mất dữ liệu.
8. **Prototype (bước 13–14).** 2 tenant với dữ liệu mẫu và công thức điểm khác nhau; test tự động cố ý truy cập dữ liệu tenant khác (kỳ vọng bị chặn); một plugin điểm danh hoặc thông báo cài/gỡ không cần deploy lại core; load test nộp bài đồng thời và test restart server giữa giờ thi.

Prompt AI gợi ý: "Giải thích PostgreSQL Row-Level Security cho multi-tenant SaaS, kèm ví dụ policy và các cạm bẫy." · "So sánh cách thiết kế plugin của Moodle và của VS Code; bài học nào áp dụng được cho LMS của tôi?"

Sinh viên phải tự giải thích được:

- Với mô hình đã chọn, migration schema cho 50 tenant được thực hiện như thế nào?
- "Noisy neighbor" là gì và thiết kế của nhóm ngăn nó ra sao?
- Một plugin lỗi có thể làm sập core không? Vì sao?

Lỗi thường gặp: coi multi-tenancy chỉ là thêm cột `tenant_id`; thiết kế plugin nhưng không chỉ rõ extension point và hợp đồng giao tiếp (interface).

7. Hướng dẫn riêng – Đề tài 5: AgriSense

Trọng tâm của đề tài này là **phân chia edge – gateway – cloud, hoạt động khi mất kết nối và xử lý luồng dữ liệu cảm biến**. Không bắt buộc phần cứng thật; toàn bộ thiết bị có thể giả lập bằng phần mềm.

1. **Tìm hiểu bối cảnh thực tế (bước 2–4).** Tìm hiểu quy trình tưới trong nhà màng (tài liệu khuyến nông hoặc hỏi người quen làm nông); xác định constraint: pin, băng thông 3G/4G, chi phí SIM data, nhiệt độ môi trường, năng lực công nghệ của nông dân.
2. **Scenario bắt buộc:** (a) mất mạng 24 giờ tại một vườn; (b) 5.000 thiết bị gửi dữ liệu cùng lúc sau khi mất điện diện rộng và có lại (thundering herd); (c) kẻ gian giả lập lệnh mở van; (d) thêm cảm biến của nhà cung cấp mới; (e) cập nhật firmware lỗi trên 10% thiết

bị.

3. **Phân chia trách nhiệm theo tầng (bước 8).** Lập bảng: chức năng nào chạy ở cảm biến, ở gateway tại vườn, ở cloud – và vì sao (độ trễ, phụ thuộc mạng, khả năng tính toán). Vẽ Deployment view có 3 tầng này.
4. **Giao thức và ingestion (bước 7).** So sánh MQTT, CoAP, HTTP cho thiết bị; chọn MQTT QoS level; thiết kế topic (ví dụ `htx/{farmId}/{deviceId}/telemetry`); adapter chuẩn hóa dữ liệu từ nhiều nhà cung cấp.
5. **Store-and-forward và device shadow (bước 7, 9).** Gateway đệm dữ liệu bao lâu, dung lượng bao nhiêu; khi có mạng lại thì đồng bộ theo thứ tự nào; xử lý xung đột khi nông dân đổi luật tưới trên app trong lúc gateway offline (desired vs reported state).
6. **Lưu trữ và xử lý dữ liệu (bước 8).** Chọn time-series DB (TimescaleDB, InfluxDB); chính sách downsampling và retention; xử lý cảnh báo bằng stream processing hay rule engine.
7. **ADR gợi ý:** giao thức thiết bị; vị trí chạy luật tưới; time-series database; cơ chế xác thực thiết bị (X.509 / token per device); chiến lược OTA (canary rollout, rollback).
8. **Prototype (bước 13–14).** Simulator ≥ 1.000 cảm biến publish lên MQTT broker (Mosquitto/EMQX); gateway giả lập chạy luật tưới cục bộ và bộ đệm; dashboard (Grafana hoặc tự viết); demo ngắt kết nối gateway–cloud 5 phút: van vẫn đóng/mở theo luật, sau khi nối lại đếm đủ số bản tin, không mất dữ liệu; demo lệnh không có chữ ký/credential hợp lệ bị từ chối.

Prompt AI gợi ý: “Giải thích MQTT QoS 0, 1, 2 và retained message; với dữ liệu độ ẩm đất và lệnh mở van, nên dùng QoS nào?” · “Device shadow của AWS IoT hoạt động thế nào; tôi có thể tự hiện thực phiên bản đơn giản ra sao?”

Sinh viên phải tự giải thích được:

- Vì sao luật tưới không thể chỉ chạy trên cloud?
- Khi 5.000 thiết bị kết nối lại cùng lúc, thiết kế của nhóm tránh quá tải bằng cách nào (jitter, backoff, rate limit)?
- Nếu gateway bị đánh cắp, kẻ gian làm được gì và thiết kế hạn chế thiệt hại ra sao?

Lỗi thường gặp: vẽ kiến trúc như một web app thông thường và bỏ qua tầng gateway; lưu dữ liệu cảm biến vào bảng quan hệ không có retention.

8. Checklist trước khi nộp và câu hỏi vấn đáp mẫu

Checklist (nhóm tự đánh dấu trước mỗi mốc)

- [ ] Mọi QA scenario đều có response measure bằng con số và nguồn gốc con số đó.
- [ ] Mọi pattern/tactic/công nghệ quan trọng đều truy vết được về ít nhất một ASR.
- [ ] Mỗi ADR có ít nhất 2 phương án được cân nhắc và có hệ quả tiêu cực.

- [ ] Các view nhất quán với nhau (tên thành phần, mũi tên, giao thức); mỗi sơ đồ có legend.
- [ ] Cấu trúc code của prototype ánh xạ được về Container/Component view.
- [ ] Số liệu kiểm chứng là số đo thật, kèm mô tả môi trường và cách tái hiện.
- [ ] AI log đã cập nhật đầy đủ, cột “giữ / sửa / bỏ và vì sao” không để trống.
- [ ] Mỗi thành viên đã đọc toàn bộ bài nộp và từng trình bày lại cho nhau phần mình không phụ trách.
- [ ] Nhóm đã luyện vấn đáp với AI hoặc với nhóm khác ít nhất một lần.

Câu hỏi vấn đáp mẫu (áp dụng cho mọi đề tài)

Cấp độ	Câu hỏi	Giảng viên kiểm tra điều gì
Nhớ – hiểu	Tactic X là gì? Nó thuộc nhóm QA nào?	Nắm khái niệm, không học vẹt
Áp dụng	Chỉ vào sơ đồ: tactic X được hiện thực ở đâu?	Liên kết lý thuyết với thiết kế của nhóm
Phân tích	Nếu bỏ thành phần Y, scenario nào thất bại và thất bại như thế nào?	Hiểu vai trò từng thành phần
Đánh giá	Vì sao chọn phương án A mà không phải B? Trade-off nhóm chấp nhận là gì?	Tự biện luận, không phụ thuộc AI
Sáng tạo	Yêu cầu mới: tải tăng 10 lần / thêm chức năng Z. Kiến trúc thay đổi gì?	Vận dụng linh hoạt, thật sự làm chủ thiết kế
Code	Đoạn code này làm gì? AI viết hay em viết? Em đã sửa gì?	Trung thực và hiểu code đã nộp

Câu hỏi “Sáng tạo” là câu phân biệt rõ nhất giữa nhóm tự thiết kế và nhóm sao chép output AI; hãy luyện kỹ dạng câu này.